Universität Basel

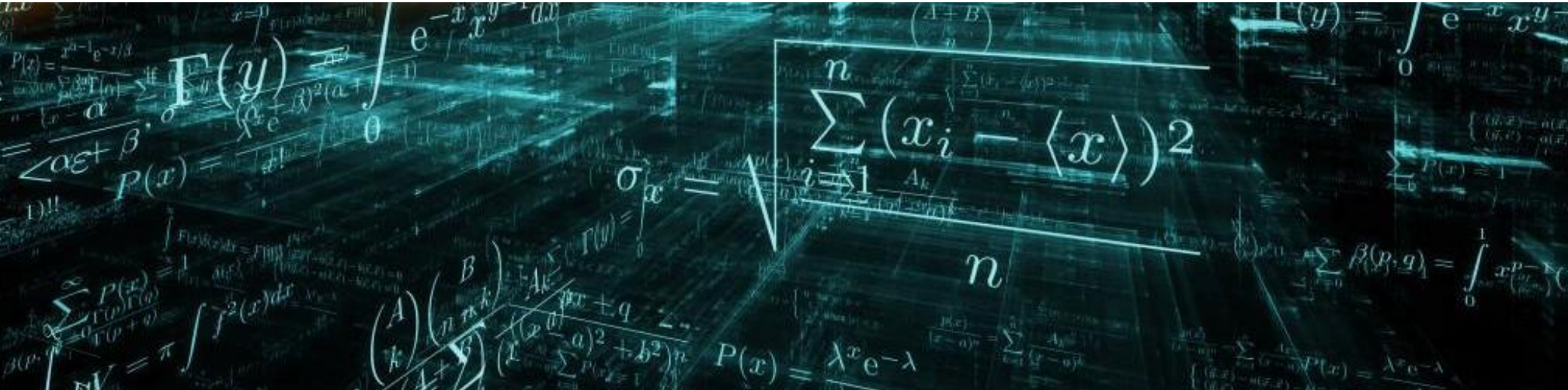
Computer Science / 15731-01 / 2026

Multimedia Retrieval

Chapter 8: Web Search

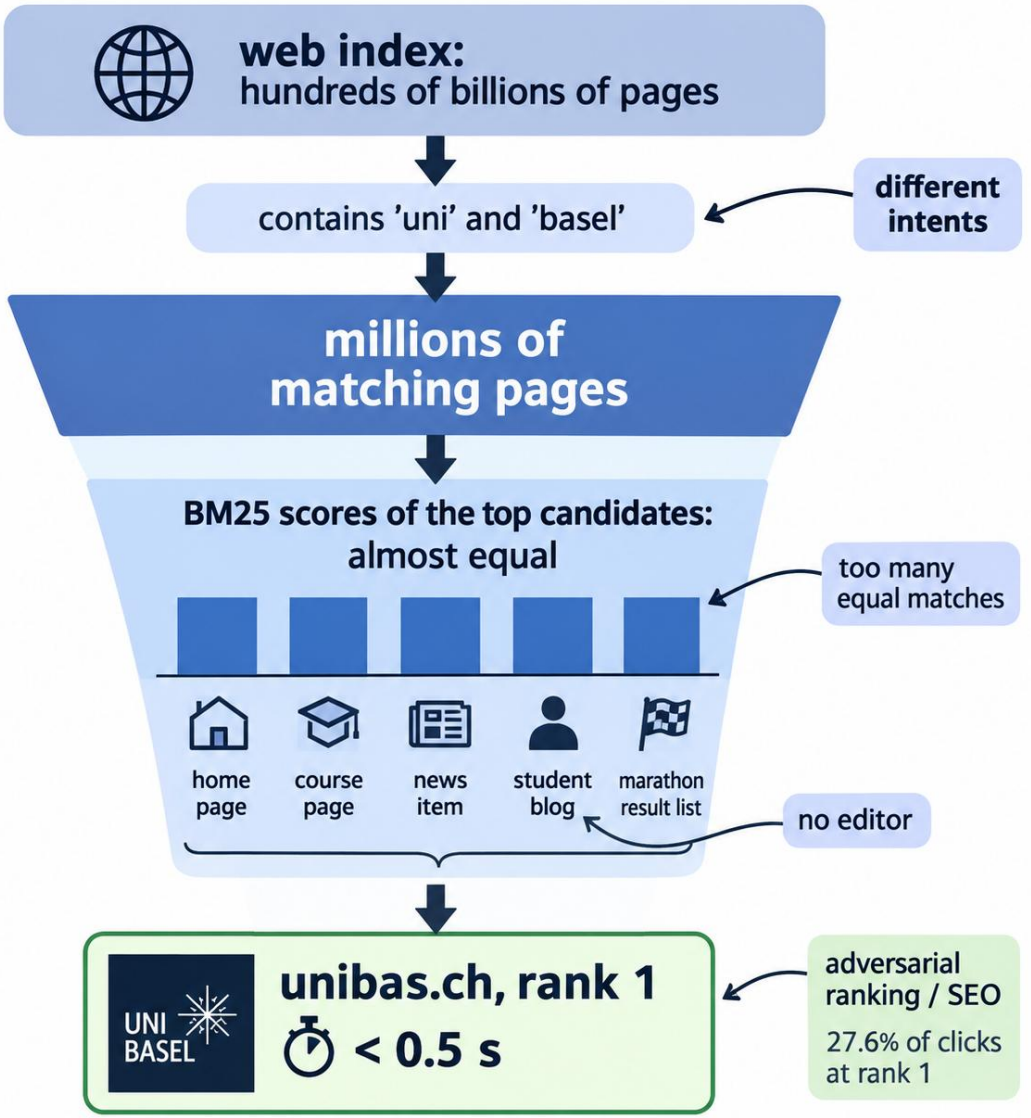
8.1 The Web Search Landscape
8.2 Ranking Signals Beyond Term Counts
8.3 Link Analysis: PageRank and HITS
8.4 Learning to Rank
8.5 Ranking Without a Query: Feeds

Dr. Roger Weber, roger.weber@gmail.com



"uni basel": the words cannot decide

- Two words discard almost the whole web, but **millions of pages** still match
  - near-equal term scores; the user expects the home page at rank 1, instantly
- **Different intents, one list**: "instagram" vs "Who was Albert Einstein?"
  - no context: serve what most users mean
- **No editor**: outdated pages, false information, scams, many copies
- **Adversarial ranking**: rank 1 gets 27.6% of clicks, page two 0.63%
  - SEO: every rewarded signal invites a fake



How web search differs from classical retrieval

- Classical retrieval assumes every document **deserves to be found** and **describes itself honestly**; not on the web

- **Low quality**: outdated pages, false information, scams, clickbait
- **Many copies**: mirrored manuals, docs, one article under several URLs

- **SEO**: legitimate (titles, clean HTML, good content) vs manipulation (stuffing, hidden text, link farms, cloaking)
  - so links, anchors, behaviour outweigh the page text
- **Deep web**: logins, database results, JavaScript apps

aspect	classical retrieval	web search
collection	controlled, quality-assured (catalogue, library)	no quality control: reference works to errors, propaganda, spam
documents	shared structure, rich metadata, facets	heterogeneous formats (HTML, PDF, scripts); little reliable structure beyond title, URL, text
queries	narrow context (artists, albums, songs)	any topic, two or three words; the intent must be inferred
results	manageable sizes, filters, facets, sorting	billions of matches for a short query; ranking decides everything
relationships	curated relations, little extra information	huge link graph showing what people consider important

What users search for: intent

- **Broder's query types**: navigational, transactional, informational
  - a coarse version of the routing intents of chapter 3
- **Ambiguous queries**: "apple", "jaguar"
  - rank for what **most users** mean, learned from clicks
  - **diversification**: rarer readings a few positions down
- **Frequent navigational queries**: served from a query-target table
  - official site not at rank 1 = failure

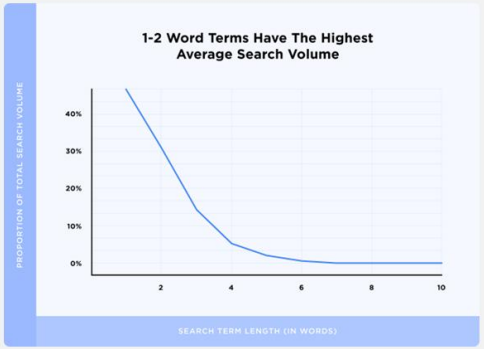
type	example	good result	routed to
navigational	"youtube", "uni basel", "facebook login"	exactly one link, at rank 1	web index or query-target table
informational	"Who was Albert Einstein?", "where to swim near Basel"	the answer on the result page	people search, map search
transactional	"new coffee machine", "trip to Hawaii"	products or offers to compare	product search

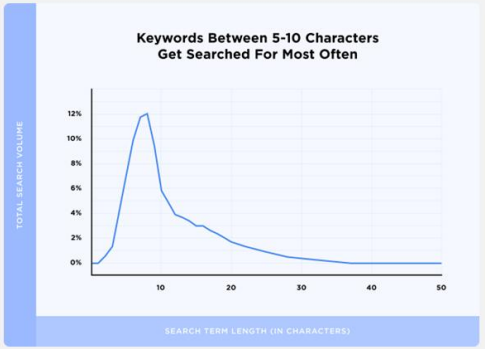
Short queries and the long tail

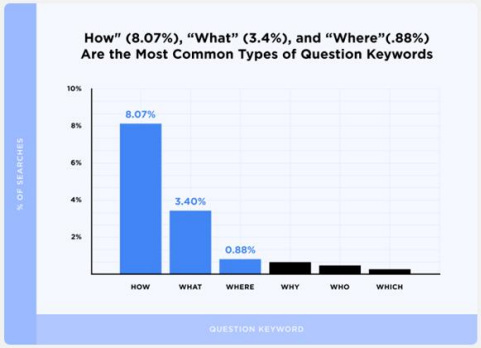
- **Short**: mostly one to three words, ten to twenty characters

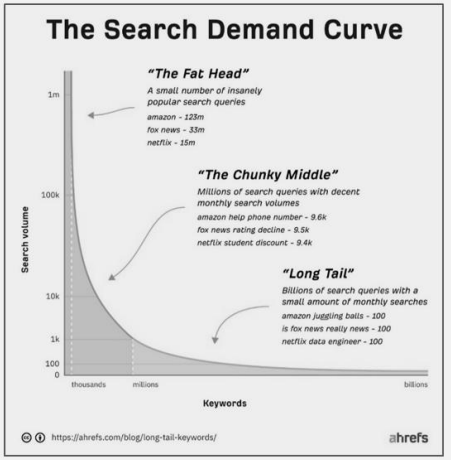
- **Zipf's law**: fat head, chunky middle, long tail
  - long tail: ≤ ~100 searches per month, 91.8% of all queries
  - top 500 queries: 8.4% of all searches

- **The tail is where engines compete**
  - head: learn the answer from clicks, or store it
  - ~15% of daily queries never seen before
  - ~14% are questions; "how" leads









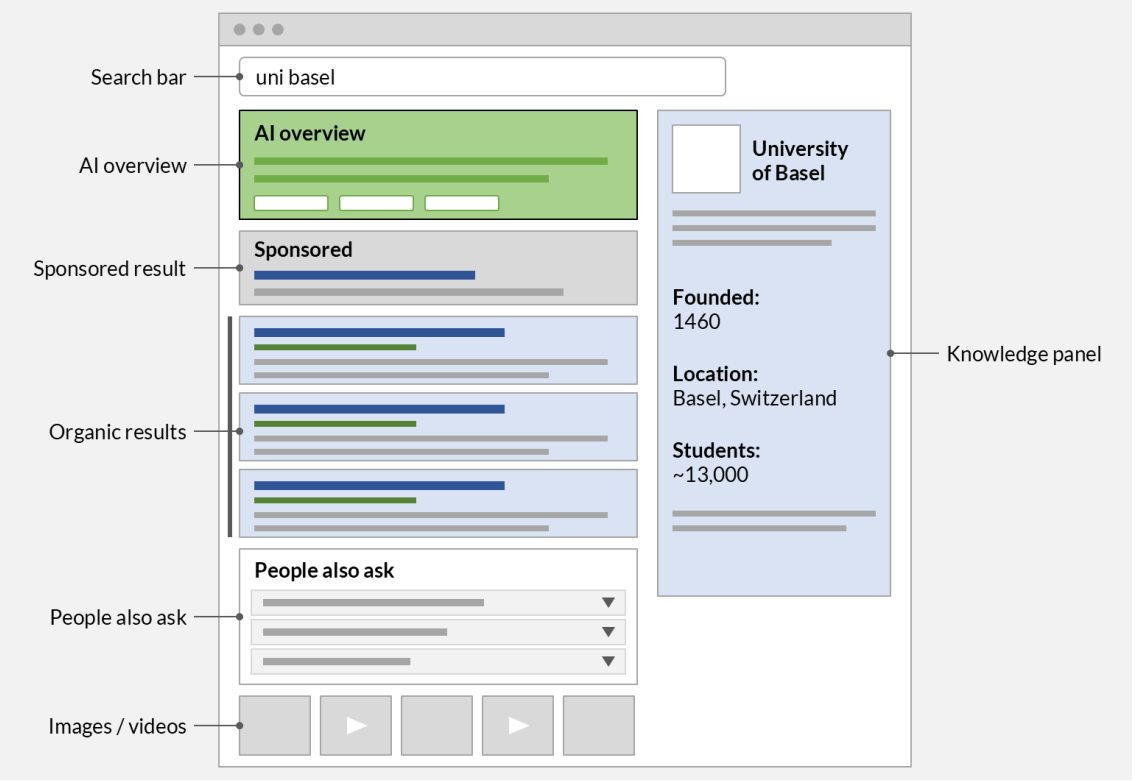
From ten blue links to a composed result page

- **Early**: one retriever over an inverted index, ten blue links (chapter 4)

- **Modern**: detect intent (chapter 3), query specialized searches, compose the page
  - learned ranker (8.4) + Knowledge Graph facts card (2012)
  - featured snippets (2014): retriever-reader (chapter 7)
  - generated answers (2023/24): RAG (chapter 7)

- Which elements, in which order: itself **a ranking problem**
  - organic results: what this chapter's signals decide

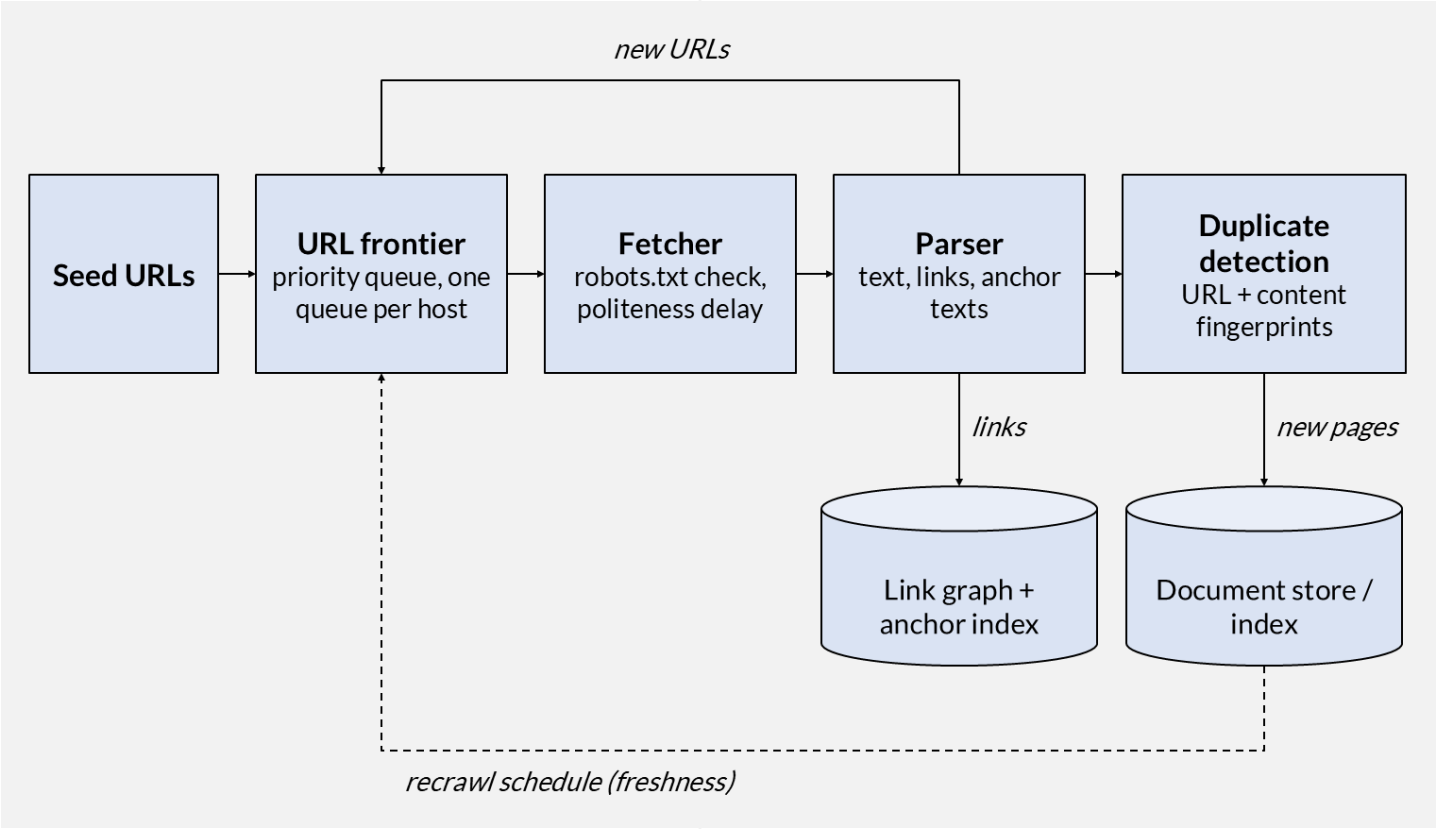
- **Zero-click** ~59%; **GEO**: tune pages to be cited



Crawling: an independent view of the web

- Sitemaps are **the site's own claim**; the crawl is an **independent view**
  - who links to a page, with which anchors
  - content as delivered; duplicates; change rates
  - the corpus behind snippets and generated answers

- **Loop**: frontier, fetcher, parser, duplicate detection (hash, SimHash)
- **Politeness**, **freshness**, **robustness**
- **robots.txt**: a request, not access control



Term proximity: hit lists and pairs

- "roger federer": a marathon list with a Roger in row 12 and a Federer in row 4,000 matches, but answers nothing
  - n-grams (chapter 3) cover only a fixed phrase list

term	positions in D
"roger"	1, 532
"federer"	2, 57, 119, 165, 184, 235, 279, 337, 373, 419, 472, 509, 533, 560, 619

- **Positional index**: each posting stores all positions, the **hit list**

```
merged hit list, sorted by position:
  roger 1, federer 2, federer 57, ...,
  federer 509, roger 532, federer 533,
  federer 560, federer 619

pairs of neighbouring hits, different terms:
  (1, 2)       distance  1
  (509, 532)   distance 23   reversed order
  (532, 533)   distance  1
```

- **Pairing rule** (Büttcher et al.): merge by position, pair **neighbouring hits of different terms**
  - 3 pairs instead of 30; surname-only mentions form none
  - no window size; any number of terms

Proximity score: bins and weights

- **Bins** (Brin and Page): bin i counts the pairs at distance i ≤ 10; weights decrease with distance

$$score_{prox}(Q,D) = \sum_{i=1}^{m} bin_i\,(Q,D) \cdot w_i \qquad with\, w_1 > w_2 > \cdots > w_m$$

- **Article**: two pairs at distance 1: score 178
- **Marathon list**: 11 pairs, all far apart: score 0

i	1	2	3	4	5	6	7	8	9	10
bin	2	0	0	0	0	0	0	0	0	0
w	89	55	34	21	13	8	5	3	2	1

- **Production**: in-order above reversed, sentence boundaries, smooth $1/d^2$
- **Price**: a larger positional index; also answers phrase queries

$$score_{prox} = 2 \cdot 89 = 178$$

▶ **Demo**: ch08-01-web-ranking-signals.ipynb

BM25F: weight, then saturate once

- HTML carries intent: **title**, **h1**, **bold**
  - home page: "university" 9 times; weighted 13 + 10 + 4 + 3 + 0.5 = 30.5
- **Naive**: saturate each field, then add
  - a spammer collects the saturated score **once per field**
- **BM25F**: weighted tf over all fields, **then** one saturation
  - k1 = 1.2, b = 0, no IDF: 30.5 gives 2.117
  - 40 stuffed occurrences gain only 0.02
  - per-field: stuffed page 41.3 instead of 2.188

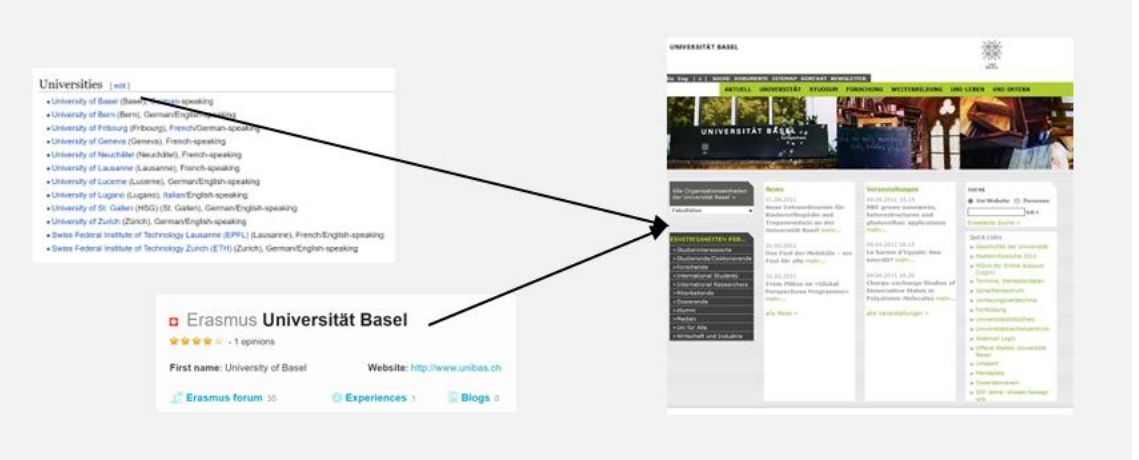
$$\widetilde{tf}(t,D) = \sum_f w_f \cdot tf_f(t,D)$$

$$sim_{BM25F}(Q,D) = \sum_{t \in Q} idf(t) \cdot \frac{\widetilde{tf}(t,D) \cdot (k_1 + 1)}{\widetilde{tf}(t,D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{avgdl}\right)}$$

page	occurrences of "university"	weighted tf	term score
University home page	9, over title, headings, body	30.5	2.117
Blog post	9, all in paragraphs	9	1.941
Keyword-stuffed page	40, all in paragraphs	40	2.136

Anchor text: other people's words, one vote per domain

- **Anchor text** describes the **target** in someone else's words
  - hard to manipulate; describes image-only and unfetched pages
- **Anchor index**: the crawler files each anchor under the target URL
  - score log(1 + n) for n votes: more votes keep helping, only slower
- **Google bomb**: "miserable failure" to a Bush biography (2003/04)
  - countermeasure: **one vote per domain**



log(1 + n)	home page	spam page
all anchors	log(1 + 12,000) = 9.39	log(1 + 50,000) = 10.82
one vote per domain	log(1 + 900) = 6.80	log(1 + 3) = 1.39

Many more signals: location, language, quality

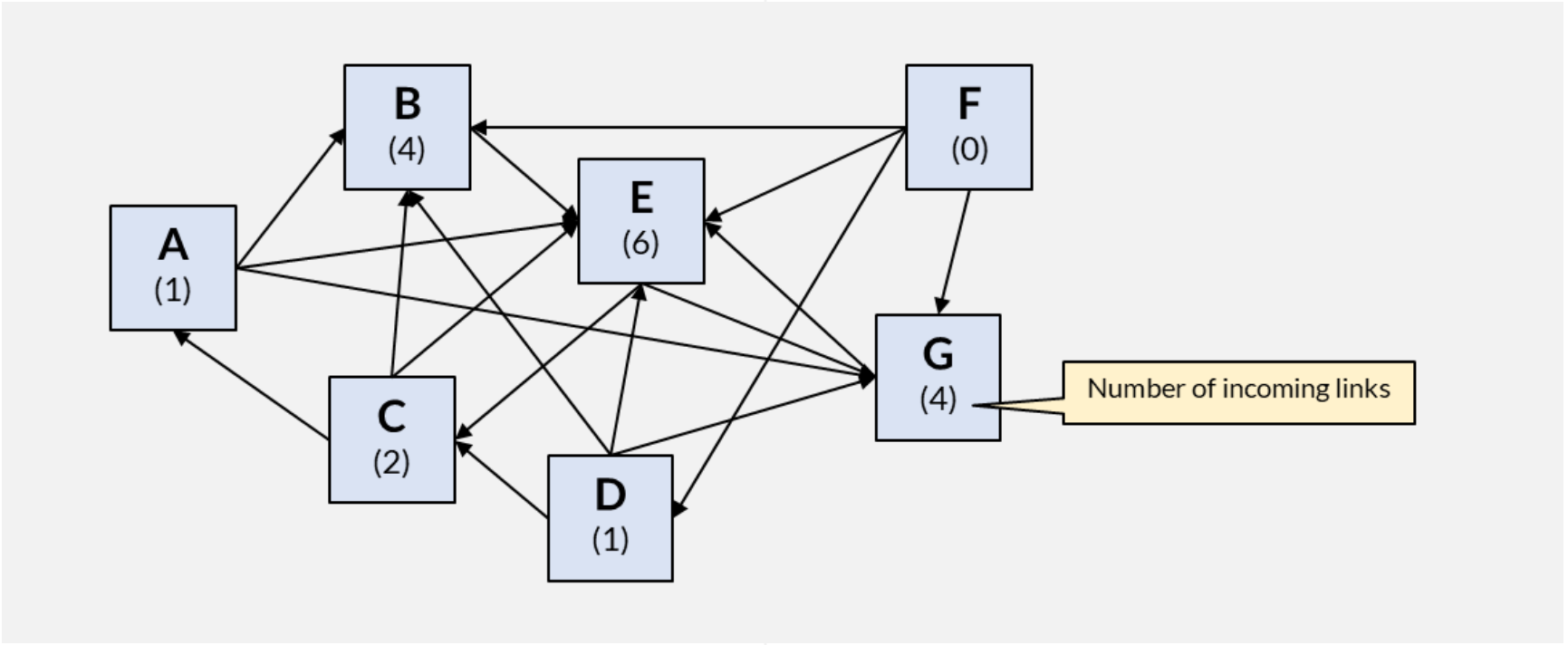
- Beyond proximity, fields, and anchors, engines use **many more signals**, for example
  - **location**: "best pizza" on a phone in Basel wants walking distance; a place in the query wins ("best pizza in Paris")
  - **language**: page language detected at indexing time; preferred, not filtered (an English page can still win for a framework error message)
  - **quality and spam**: penalties for thin, copied, or stuffed pages; link farms and malware removed
  - **site trust**: ownership, age, reputation; human raters judge E-E-A-T, their labels train the rankers
  - **freshness**, URL depth, click rates for similar queries, neural scores, ...

- Each signal is simple on its own; the hard part is **combining hundreds of them**: learning to rank (8.4)

- **Cost of trust**: an excellent new site struggles to reach the top; balance trust against freshness and diversity

Counting links is not enough

- A link is an **implicit recommendation**; the graph's topology shows what people consider important
- **In-degree**: E 6, B and G 4, C 2, A and D 1, F 0
- **Problem 1**: B and G tie, but only G is linked by E, the most important page
- **Problem 2**: **link farms** inflate in-degree at almost no cost
- Wanted: a link worth only as much as its source



PageRank: the random surfer and power iteration

- **Random surfer**: with probability α follow a random link, else jump to a random page; dangling nodes always jump
- **PageRank**: the stationary distribution of this Markov chain
- **Fixes in-degree**: a link passes PR(q), split over L(q) outlinks; link farms gain almost nothing
- **Random jump**: no rank sinks, a unique solution
- **Power iteration**: start uniform, repeat until stable
  - one sparse matrix-vector product per step; 322 M links in ~52 iterations

$$PR(p) = \frac{1-\alpha}{N} + \alpha \cdot \sum_{q \to p} \frac{PR(q)}{L(q)} \qquad \forall p \in \mathbb{P}$$

$$M_{i,j} = \begin{cases} \frac{1}{L(p_j)} & if\, p_j \to p_i \\ \frac{1}{N} & if\, p_j \text{ has no outgoing links} \\ 0 & otherwise \end{cases}$$

$$\mathbf{r} = \frac{1-\alpha}{N} \cdot \mathbf{1} + \alpha \cdot \mathbf{M}\mathbf{r}$$

$$\mathbf{r}^{(0)} = \frac{1}{N} \cdot \mathbf{1} \qquad \mathbf{r}^{(t+1)} = \frac{1-\alpha}{N} \cdot \mathbf{1} + \alpha \cdot \mathbf{M}\mathbf{r}^{(t)}$$

PageRank on the seven-page web

- α = 0.85, N = 7, all pages start at 1/7 = 0.143

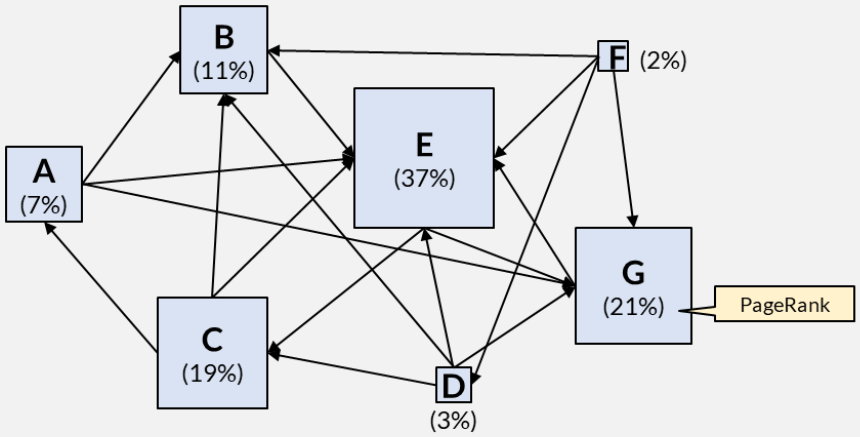
- **First step for E**: links from A (3 outlinks), B (1), C (3), D (4), F (4), G (1)
- **F**: no inlinks, only the jump share 0.021
- Converged after 36 iterations (10^{-8})

- **G and C overtake B**
  - E has two outlinks: C and G each get 42.5% of its PageRank
  - B's inlinks come from weak pages

$$PR^{(1)}(E) = \frac{0.15}{7} + 0.85 \cdot \frac{1}{7}\left(\frac{1}{3} + 1 + \frac{1}{3} + \frac{1}{4} + \frac{1}{4} + 1\right)$$

$$PR^{(1)}(E) = 0.021 + 0.85 \cdot 0.452 = 0.406$$

page	A	B	C	D	E	F	G
inlinks	1	4	2	1	6	0	4
PR	7.4%	10.5%	18.6%	2.6%	37.5%	2.1%	21.2%



The random jump: how often, and where to

- **α**: small, all pages drift to 1/N; near 1, links dominate and convergence slows
- **Not relevance**: query-blind prominence, one feature among many
  - favours old pages: news needs freshness
- **Personalized PageRank**: jump only to a set S
  - S = a topic: topic-sensitive PageRank
  - S = one user: "who to follow"

page	α = 0.50	α = 0.85	α = 0.95
A	9.7%	7.4%	7.0%
B	13.2%	10.5%	9.6%
C	15.3%	18.6%	19.8%
D	8.0%	2.6%	0.9%
E	28.7%	37.5%	39.8%
F	7.1%	2.1%	0.7%
G	17.8%	21.2%	22.2%
iterations to 10^{-8}	17	36	47

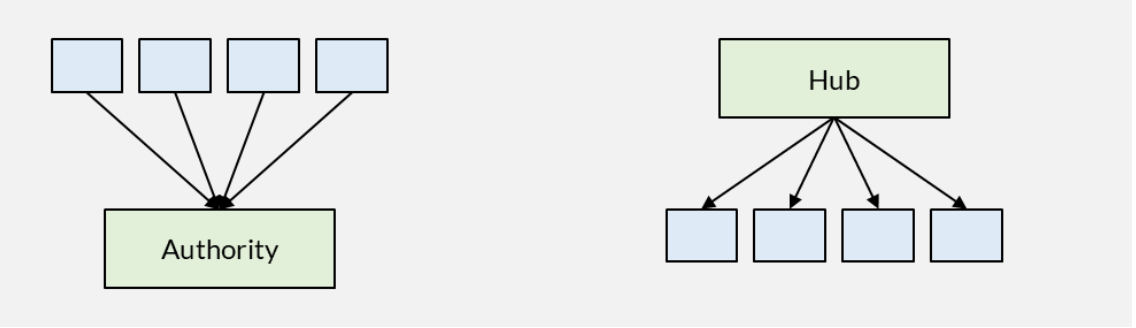
$$\mathbf{r} = (1 - \alpha) \cdot \mathbf{s} + \alpha \cdot \mathbf{M}\mathbf{r}$$

s: jump distribution, non-zero only on the jump set S

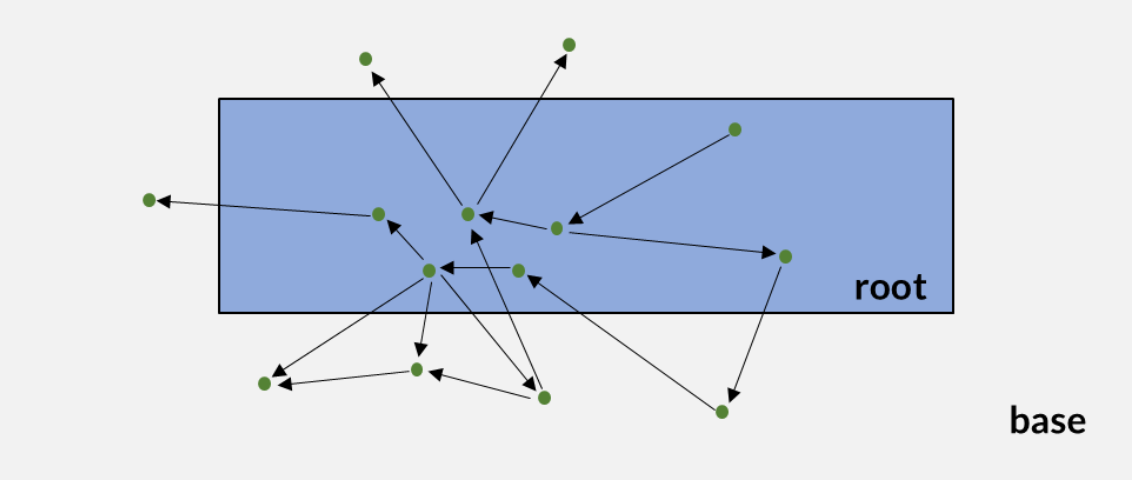
▶ **Demo**: ch08-02-link-analysis.ipynb

Hubs and authorities: HITS

- **Authority**: a definitive source; receives many links
- **Hub**: a curated list; links to many authorities
- Roles reinforce each other, **for one topic**



- **Query graph**: root set = top ~200 text results
  - base set: plus pages they link to and pages linking to them
  - links within a domain dropped
  - inlinks need a crawl-time link index

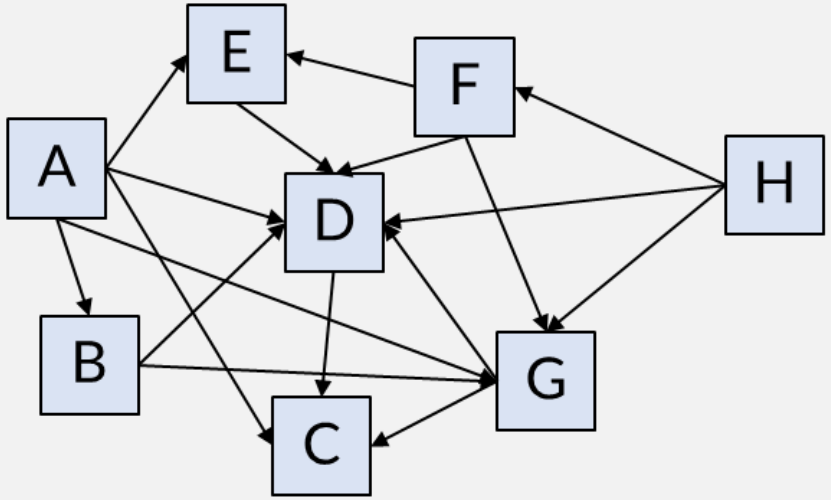


HITS: iteration and example

- **Update**: authority = sum of hubs linking in; hub = sum of authorities linked to; normalize
- **A** best hub (0.60), **D** best authority (0.69)
- **E (0.31) > C (0.29)** with fewer inlinks: linked by the strong hubs A and F

$$a^{(t+1)}(p) = \sum_{q \to p} h^{(t)}(q) \qquad h^{(t+1)}(p) = \sum_{p \to q} a^{(t)}(q)$$

$$\sum_p a(p)^2 = \sum_p h(p)^2 = 1$$



Adjacency Matrix

	A	B	C	D	E	F	G	H		*h*	*a*
A		1	1	1	1		1			60%	0%
B				1			1			36%	18%
C										0%	29%
D			1							9%	69%
E				1						20%	31%
F				1	1		1			46%	12%
G			1	1						29%	54%
H				1		1	1			40%	0%

▶ **Demo**: ch08-02-link-analysis.ipynb

PageRank vs HITS

- **PageRank**: offline, whole web, global prominence
- **HITS**: at query time, a few thousand pages, one topic

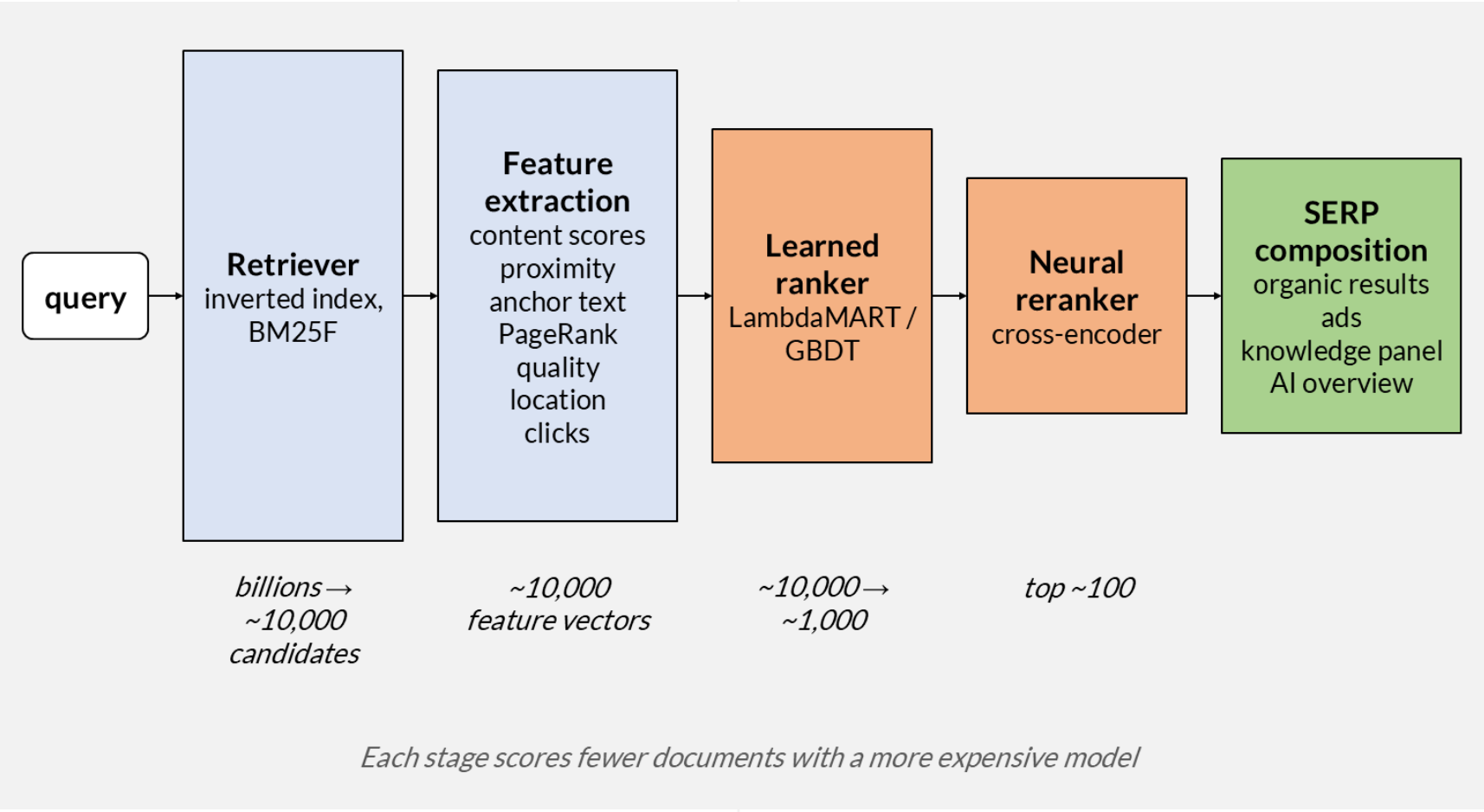
- Search engines adopted PageRank; hubs and authorities live on in SALSA (Twitter's Who to Follow)

- **HITS weaknesses**: per-query cost; dense off-topic groups take over; domain-wide links; auto-generated links; "jaguar car" drifts to cars

model	computed	scope	typical use
In-degree	offline	global	naive baseline; easy to manipulate
PageRank	offline, once per crawl	global, query-independent	page-level prominence feature
Personalized PageRank	offline or per user	relative to a node or topic	topic-sensitive ranking, who to follow
HITS	at query time	one topic, query base set	topic distillation; hub and authority roles

The ranking stack

- **Hundreds of signals** must become **one score per page**
- This section: the **learned ranker**; one feature vector in, one score out
- **Stages**: each passes fewer candidates to a costlier model



Combining signals by hand

- "uni basel": four candidates, four signals, grades 0 to 3
  - ideal d1, d4, d2, d3; BM25F alone: nDCG 0.817
- **Weighted sum** by hand
  - proximity does not help
  - PageRank or anchor text fixes the ranking
- **Three different weight vectors are perfect**: one query cannot tell them apart
- Weights must work over **thousands of queries and hundreds of signals**: **learning to rank**

page	BM25F	prox.	anchor	PR	grade
d1: home page	2.1	0.9	6.8	0.9	3
d2: course page	2.3	0.8	1.1	0.2	1
d3: marathon list	1.9	0.1	0.0	0.1	0
d4: Wikipedia	2.0	1.0	4.5	0.7	2

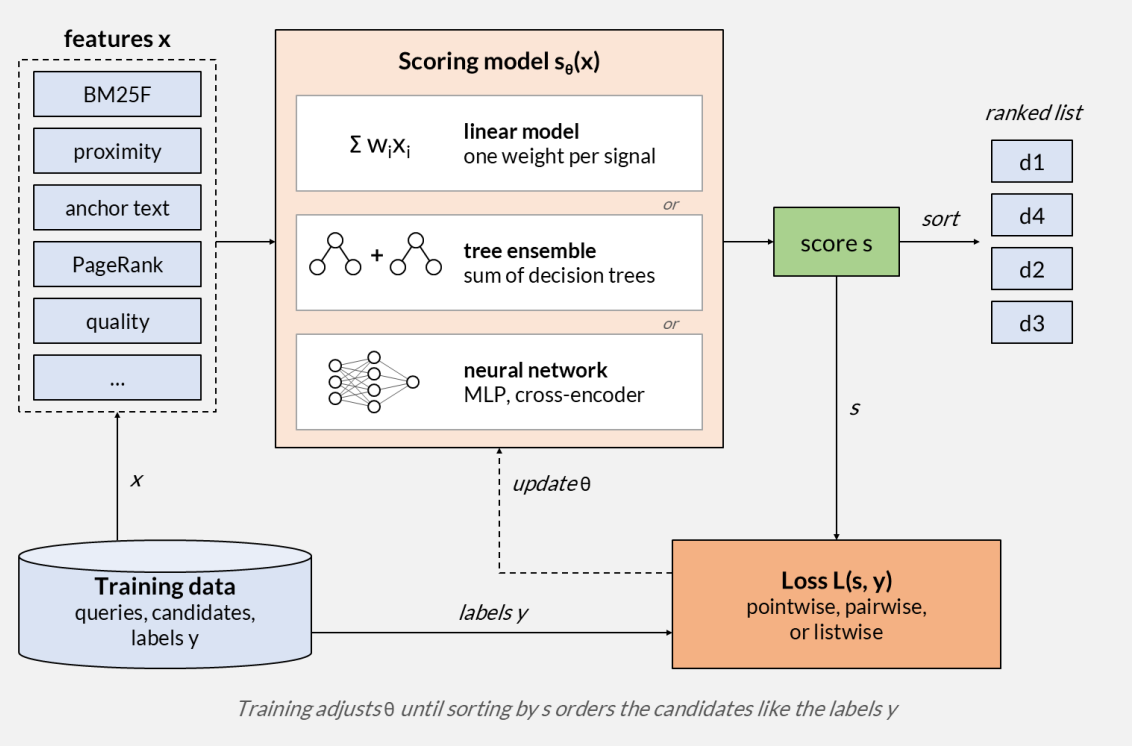
weights w	scores d1 to d4	ranking	nDCG
(1, 0, 0, 0)	2.10, 2.30, 1.90, 2.00	d2, d1, d4, d3	0.817
(1, 0.5, 0, 0)	2.55, 2.70, 1.95, 2.50	d2, d1, d4, d3	0.817
(1, 0, 0, 1)	3.00, 2.50, 2.00, 2.70	d1, d4, d2, d3	1
(1, 0.5, 0.3, 1)	5.49, 3.23, 2.05, 4.55	d1, d4, d2, d3	1
(0, 0, 1, 0)	6.80, 1.10, 0.00, 4.50	d1, d4, d2, d3	1

The learning-to-rank problem

- **Scoring model** s_θ: features x to a score s; sort by s

- **Training data**: queries, candidates, features x, labels y
  - query-dependent: BM25F, proximity, anchors, embeddings
  - query-independent: PageRank, quality, length, age
  - query features: length, intent, place
  - labels: human grades or clicks

- **Loss** per query: only the order within a query matters



$$\theta^* = \operatorname{argmin}_\theta \sum_Q \mathcal{L}\left(\mathbf{s}_Q, \mathbf{y}_Q\right) \qquad \mathbf{s}_Q = \left(s_\theta(\mathbf{x}_{Q,D_1}), \ldots, s_\theta(\mathbf{x}_{Q,D_n})\right)$$

The scoring model: linear, trees, neural

- **Linear**: cheap, readable; no interactions ("PageRank only if the page matches")

$$s_\theta(\mathbf{x}) = \mathbf{w}^\top \mathbf{x}$$

- **Tree**: threshold questions; three reproduce all four grades
  - **GBDT**: hundreds of trees, each corrects the previous
  - needs features **normalized per query**

```
anchor > 3 ?
  yes:  PageRank > 0.8 ?
          yes -> 3   (d1)
          no  -> 2   (d4)
  no:   proximity > 0.5 ?
          yes -> 1   (d2)
          no  -> 0   (d3)
```

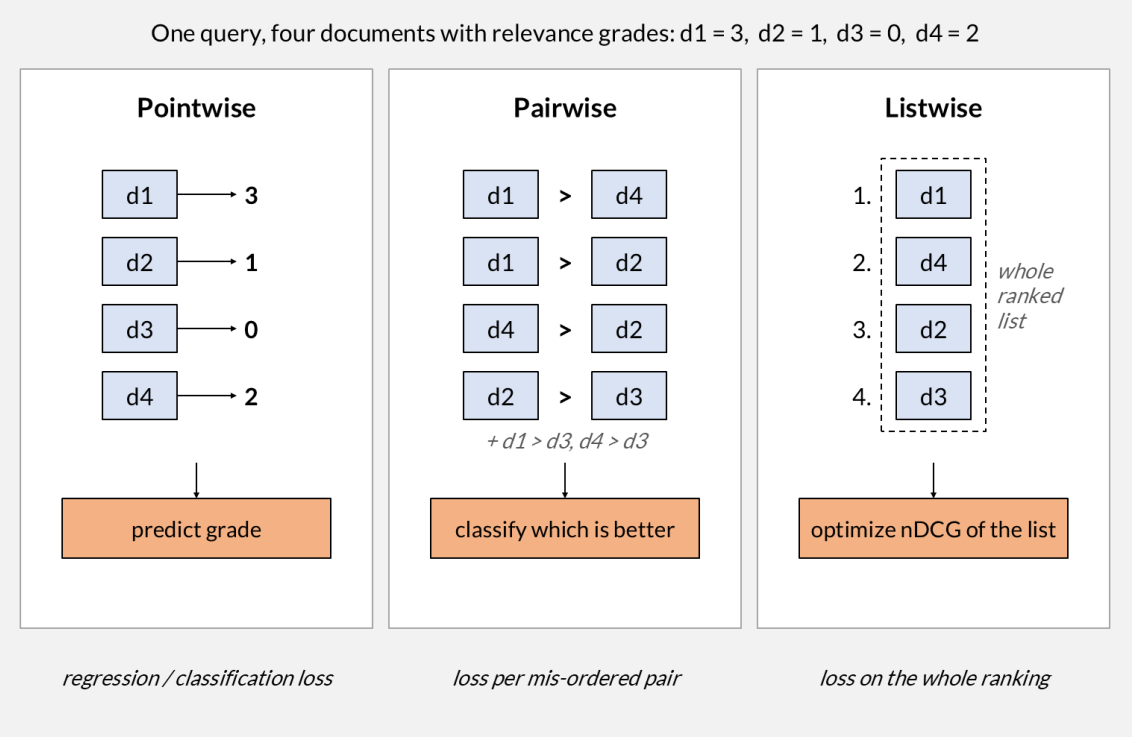
- **Neural**: MLP; a cross-encoder reads the text itself
- Trees in the middle of the stack, cross-encoders at the top

$$s_\theta(\mathbf{x}) = \sum_{m=1}^{M} f_m(\mathbf{x})$$

$$s_\theta(\mathbf{x}) = \mathbf{w}_2^\top \, ReLU(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

Three ways to learn a ranking: pointwise first

- Families differ in the **unit of training**: one document, one pair, one list
  - not model types: any model with any loss
- **Pointwise**: predict the grade per (Q, D)
  - linear fit on four pages: exact, but weights (0.13, 0.65, 0.73, −3.16)
- **Wrong goal**: d1 at 4.0 instead of 3 costs 1.0, harmless; d4 at 1.6, d2 at 1.7 cost 0.65 and swap two pages
- **Grades not comparable across queries**



$$\mathcal{L} = \sum_{D} (s_\theta(\mathbf{x}_{Q,D}) - y_{Q,D})^2$$

Pairwise: order each pair (RankNet)

- Every pair with different grades is one example; six pairs here

- **RankNet** (Burges et al., 2005): logistic on the score difference, cross-entropy over pairs
  - **gradient**: each pair pulls w along x_i − x_j, strongly if mis-ordered
  - one step from BM25F alone: largest pull on the anchor weight; ranking now ideal

- **Strengths**: closer to the goal; many examples; clicks yield pairs
- **Weakness**: all pairs equal; ranks 1 and 2 like ranks 40 and 41

$$P(d_i \succ d_j) = \sigma(s_i - s_j) \qquad \mathcal{L} = \sum_{(i,j):\, y_i > y_j} -\log\sigma(s_i - s_j)$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = -\sum_{(i,j):\, y_i > y_j} \sigma\left(-(s_i - s_j)\right)(\mathbf{x}_i - \mathbf{x}_j) \qquad \mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{w}}$$

	weights w	scores d1 to d4	ranking	loss
BM25F alone	(1, 0, 0, 0)	2.10, 2.30, 1.90, 2.00	d2, d1, d4, d3	4.052
after 1 step	(1.003, 0.060, 0.591, 0.073)	6.24, 3.02, 1.92, 4.78	d1, d4, d2, d3	0.762
after 50 steps	(1.124 0.293 1.582 0.178)	13.54, 4.60, 2.18, 9.79	d1, d4, d2, d3	0.115

Listwise: optimize nDCG (LambdaRank)

- **Goal**: optimize nDCG of the whole list
  - nDCG jumps when two scores cross: no useful gradient
- **LambdaRank**: keep RankNet's pair gradients, scale each by **|ΔnDCG|** of swapping the pair
  - pairs near the top get the strongest updates
- **BM25F ranking** d2, d1, d4, d3
  - strongest push: d1 > d2, mis-ordered at the top
  - weakest: d1 > d4, correct at ranks 2 and 3
- **LambdaMART**: each new tree fitted to the lambdas
  - won the Yahoo! LTR Challenge (2010)

$$\lambda_{ij} = |\Delta nDCG_{ij}| \cdot \sigma\big(-(s_i - s_j)\big)$$

pair	current ranks	abs ΔnDCG	σ(−(s_i − s_j))	λ_ij
d1 > d2	2, 1	0.155	0.550	0.085
d4 > d2	3, 1	0.105	0.574	0.060
d1 > d3	2, 4	0.126	0.450	0.057
d2 > d3	1, 4	0.120	0.401	0.048
d4 > d3	3, 4	0.029	0.475	0.014
d1 > d4	2, 3	0.027	0.475	0.013

Neural rankers join learning to rank: TREC-COVID

- **Transformers since 2019**: BERT on ~1 in 10 US English queries
  - neural score: one more feature, or the final stage
- **TREC-COVID**: 50 topics, 66,336 judgments, 8 features; train 40, test 10, five rounds
- Every learned ranker beats the best single signal
- Pairwise and listwise beat pointwise; LambdaMART best
- The neural score **completes** term statistics

ranker	nDCG@10
BM25F alone (no learning)	0.660
neural score alone (no learning)	0.704
linear, pointwise (least squares)	0.765
linear, pairwise (RankNet)	0.791
trees, pointwise (regression)	0.807
trees, listwise (LambdaMART)	0.816
LambdaMART without the neural score	0.649

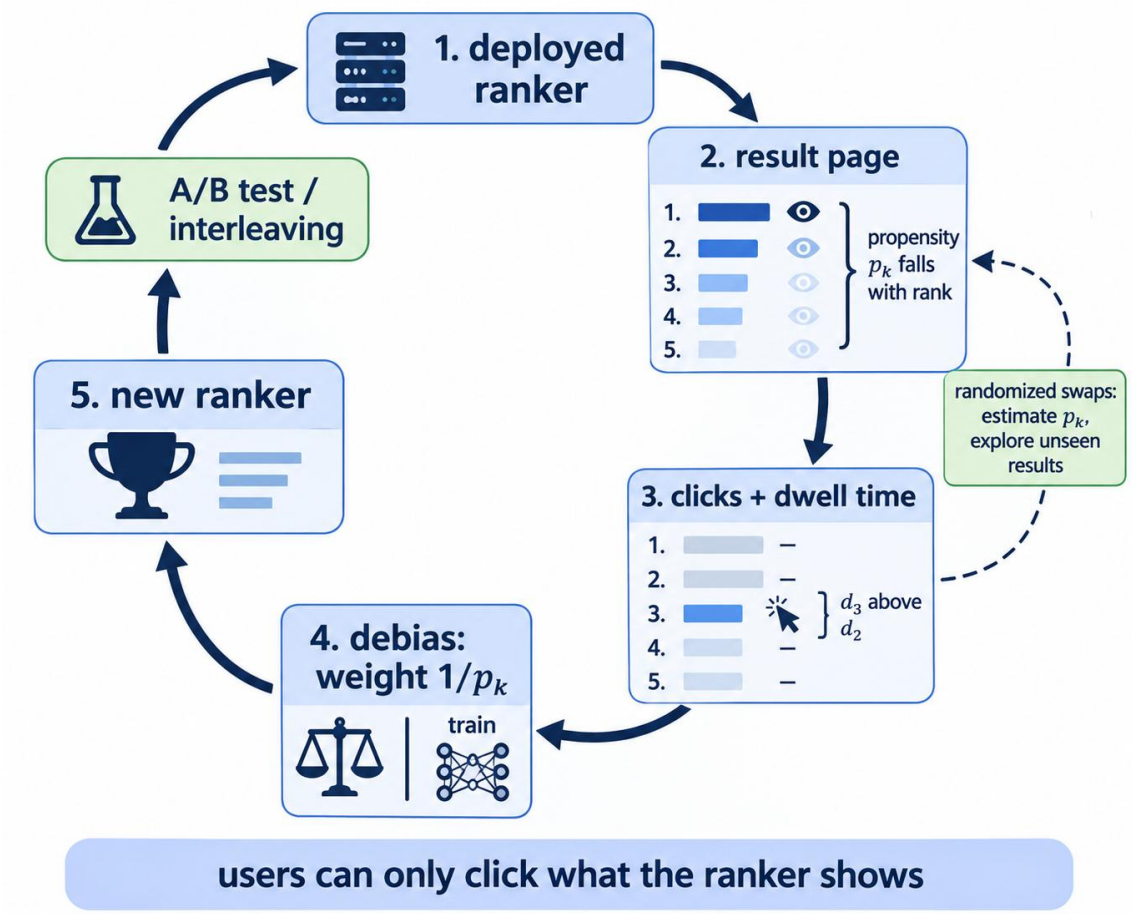
▶ **Demo**: ch08-03-learning-to-rank.ipynb

Continuous learning to rank: the training loop

- Rankers are **retrained continuously**: new pages, new queries, shifting meanings, new signals

- **Inputs**
  - **human raters**: precise, slow; test sets and hard queries
  - **user behaviour**: clicks, dwell time, quick returns to the list
- **Clicks are biased**: top ranks get clicks for being on top; clickbait gets clicks too
  - debias before training; small randomized swaps measure the bias

- **A/B test** each revision on a fraction of users: **ship or roll back**
- Blind spot: users only click what is shown



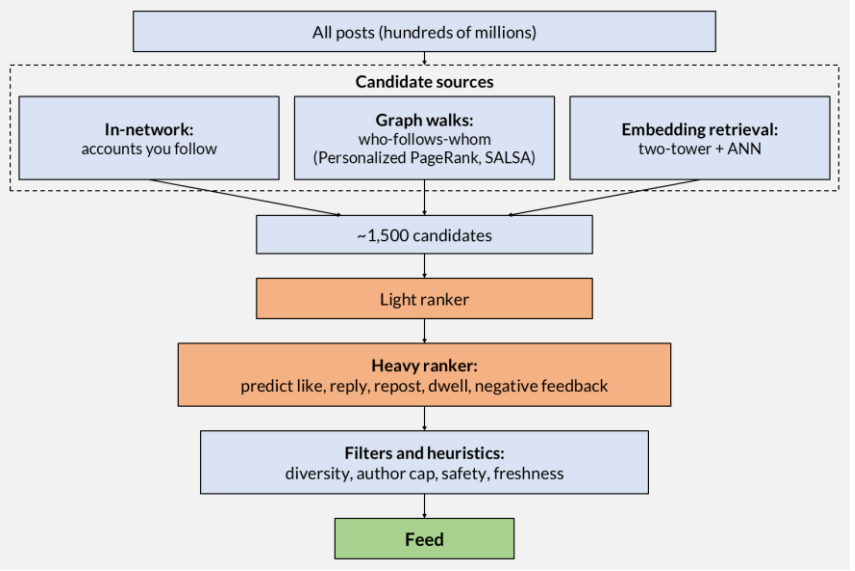
Ranking without a query: feeds

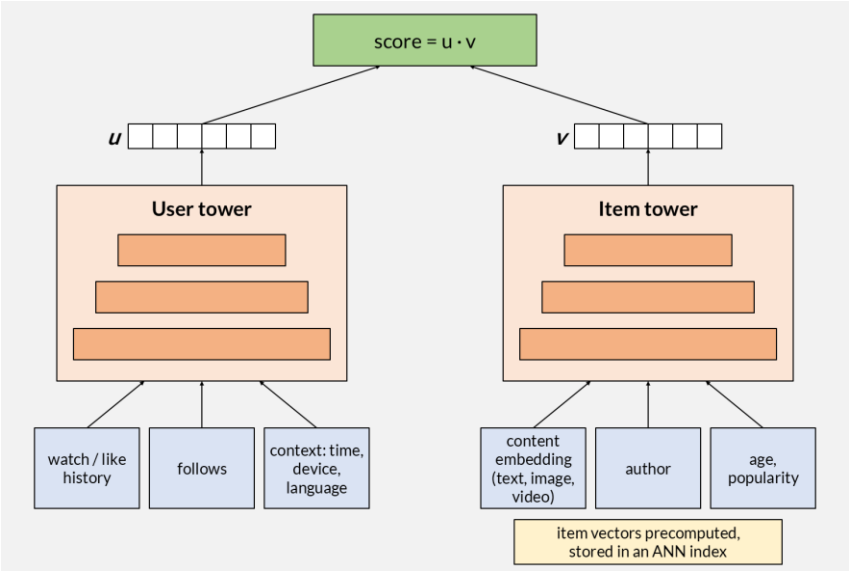
- **Push instead of pull**: TikTok, YouTube, Instagram, X show content before any query

- **The user is the query**: a user tower embeds watch and like history, follows, and context (time, device, language)
  - item tower embeds every item from its content, author, age: new videos recommendable at once
  - ANN index returns the nearest items: the bi-encoder of chapter 5

- **Social graph**: Personalized PageRank, SALSA for "who to follow"

- **Same staged design**: ~1,500 candidates, a heavy ranker predicts reactions, filters shape the feed
  - weights on the reactions are a product decision: engagement is not value





Key takeaways

- The web is uncurated and adversarial, queries are short and ambiguous, and millions of pages match equally well; the engine serves the intent most users mean, composes the result page for it, and relies on a crawler for evidence the page itself does not provide
- Content signals go beyond term counts: proximity of query terms, BM25F weights fields before one saturation, anchor text counted with one vote per domain, plus location, language, and site quality
- PageRank is the stationary distribution of a random surfer and measures global prominence, not relevance; HITS finds topic-specific hubs and authorities on a query graph
- Learning to rank combines hundreds of signals; pairwise and listwise losses target the order, and LambdaRank weights each pair by its effect on nDCG
- Clicks are abundant labels but biased by position; debias them before training, and confirm new rankers online
- Feeds rank without a query in the same staged design: embeddings and the social graph supply candidates, a learned ranker predicts reactions, and the chosen weights decide what the platform optimizes

Further reading

- Brin and Page (1998). The anatomy of a large-scale hypertextual web search engine.
- Page et al. (1999). The PageRank citation ranking. Stanford.
- Kleinberg (1999). Authoritative sources in a hyperlinked environment. JACM.
- Bharat and Henzinger (1998). Topic distillation. SIGIR.
- Lempel and Moran (2001). SALSA. ACM TOIS.
- Haveliwala (2002). Topic-sensitive PageRank. WWW.
- Büttcher et al. (2006). Term proximity scoring. SIGIR.
- Robertson et al. (2004). BM25 for multiple weighted fields. CIKM.
- Broder (2002). A taxonomy of web search. SIGIR Forum.
- Olston and Najork (2010). Web crawling. FnTIR.
- Aggarwal et al. (2024). GEO. KDD.
- Liu (2009). Learning to rank for IR. FnTIR.
- Burges et al. (2005). RankNet. ICML.
- Burges (2010). From RankNet to LambdaMART.
- Chapelle and Chang (2011). Yahoo! LTR Challenge.
- Joachims (2002). Clickthrough data for ranking. KDD.
- Joachims et al. (2005). Clicks as implicit feedback. SIGIR.
- Joachims et al. (2017). Unbiased learning-to-rank. WSDM.
- Covington et al. (2016). YouTube recommendations. RecSys.
- Yi et al. (2019). Two-tower retrieval at YouTube. RecSys.
- Gupta et al. (2013). WTF: Who to follow at Twitter. WWW.
- Twitter (2023). The recommendation algorithm.